

I 1 Giai đoạn 1: tự viết bằng NumPy (Building It Yourself in NumPy)

PHỤ THUỘC A/01-toan-toi-thieu/02-vi-phan-va-toi-uu (quy tắc chuỗi, Jacobian); B/09-thiet-ke-ham-mat-mat (*hinge loss* và *cross-entropy*); C/12-co-che-huan-luyen/01-co-hoc-cua-gradient và 02-lam-cho-training-chay-duoc (BatchNorm, dropout — ở đây ta cài lại chúng bằng tay).

TÓM TẮT

Mục này là bài tập một lần: tự cài **lan truyền ngược** (*backpropagation*) bằng NumPy cho sáu tầng, kiểm tra từng tầng bằng **kiểm tra gradient số** (*numerical gradient check*), rồi ghép thành một mạng chạy được. Nó không phải bài tập công cụ mà là bài tập trực giác tầng [T2][T3]: đọc xong bạn giải thích được vì sao bộ nhớ huấn luyện lớn hơn bộ nhớ **suy luận** (*inference*) nhiều lần, vì sao quên `eval()` lại đổi kết quả, và bạn tự chẩn đoán được một gradient sai thay vì đoán mò. Nếu chỉ nhớ một điều: không ghép một tầng chưa qua kiểm tra gradient — vì kiểm từng tầng giữ không gian nghi vấn ở đúng một phần tử, còn ghép trước kiểm sau làm nó phình lên thành số tầng nhân số tương tác.

KIẾN THỨC CẦN CÓ TRƯỚC

- **hàm mất mát (loss function)** — hàm biến đầu ra của mô hình và đáp án đúng thành một số thực đo mức sai. Huấn luyện là cực tiểu hoá số đó.
- **gradient và quy tắc chuỗi** — gradient là vector đạo hàm riêng của loss theo từng tham số. Vì mạng là hàm hợp của nhiều tầng, đạo hàm được tính bằng quy tắc chuỗi — nhân dồn các đạo hàm cục bộ từ đầu ra ngược về đầu vào.
- **kích hoạt (activation)** — đầu ra trung gian của một tầng. Lượng bộ nhớ huấn luyện phần lớn là chỗ chứa các kích hoạt này, vì lượt ngược cần đọc lại chúng.
- **ReLU, dropout, BatchNorm** — ReLU là hàm phi tuyến $\max(0, x)$; dropout tắt ngẫu nhiên một phần kích hoạt lúc huấn luyện để chống *overfitting*; BatchNorm chuẩn hoá kích hoạt theo thống kê của *batch* để việc huấn luyện ổn định hơn. Cả ba đều có hành vi *khác nhau* giữa lúc train và lúc test.
- **batch** — nhóm mẫu được xử lý cùng lúc trong một bước cập nhật. Kích thước batch ảnh hưởng tới bộ nhớ, tốc độ, và — với BatchNorm — tới cả kết quả.
- **SGD và momentum** — *stochastic gradient descent* cập nhật trọng số theo gradient của một batch; *momentum* cộng dồn hướng đi của các bước trước để giảm dao động và vượt qua vùng phẳng.
- **float32 so với float64** — hai độ chính xác dấu phẩy động. float32 có epsilon máy $\approx 1,2 \times 10^{-7}$, float64 khoảng $2,2 \times 10^{-16}$; khác biệt này quyết định việc kiểm tra gradient số có nói thật hay không.

A. Đặt vấn đề

Framework che giấu rất nhiều thứ, và phần lớn thời gian đó là điều tốt — không ai muốn viết lại conv2d mỗi sáng. Nhưng có một tập nhỏ những thứ mà nếu không tự tay làm một lần thì bạn sẽ không bao giờ debug nổi, vì triệu chứng của chúng không bao giờ xuất hiện dưới dạng một thông báo lỗi. Mạng vẫn chạy, loss vẫn giảm, chỉ là giảm chậm hơn đáng ra, hoặc kết quả lúc `eval()` lệch khỏi lúc train một cách không giải thích được. Không có exception nào để bắt, không có *stack trace* nào để đọc.

Cần nói rõ ngay từ đầu về *loại* bài tập này, vì hiểu sai loại là cách phổ biến nhất để lãng phí một tháng:

- **Đây không phải bài tập tăng bốn** — không phải bài tập công cụ. Bạn sẽ không dùng lại một dòng NumPy nào trong công việc thật, và đó là chuyện bình thường.

- Đây là bài tập tăng tầng hai và ba [T2][T3]: mục tiêu là *trực giác về cơ chế*, thứ không hết hạn khi framework đổi phiên bản, khi PyTorch bị thay bằng cái khác, hay khi kiến trúc thời thượng đổi tên.
- Làm một lần, đạt tiêu chí dừng, rồi không quay lại. Cái bấy quen thuộc của người học theo CS231n là yêu quá trình này và tiếp tục viết NumPy sau khi nó đã hết tác dụng — lúc đó nó thành một sở thích thủ công chứ không còn là học.

Học xong thì làm được gì mà trước đó không làm được? Bạn nhìn một đường cong loss bất thường và khoanh vùng được ngay nó là lỗi gradient, lỗi trạng thái hay lỗi dữ liệu; bạn đọc thông báo hết bộ nhớ GPU và biết tầng nào đang giữ activation; và bạn viết được một autograd.Function tùy biến trong PyTorch mà không phải sao chép mẫu từ diễn đàn.

Người đọc đã viết C++ và CUDA có một lợi thế ở đây, và cũng có một cái bấy riêng. Lợi thế là bạn đã quen với việc một hệ thống số học có thể sai ở mức không nhìn thấy được. Cái bấy là bạn dễ bỏ qua bước kiểm tra gradient vì thấy nó buồn tẻ. Đúng như một người viết solver phi tuyến để bỏ bước so Jacobian giải tích với Jacobian sai phân, rồi mất ba ngày cho một dấu trừ.

TRỰC GIÁC

Bạn đã làm chính xác bài tập này rồi, chỉ là ở lĩnh vực khác. Khi thêm một loại *residual* mới vào một bài toán bình phương tối thiểu phi tuyến, bạn viết Jacobian giải tích để solver chạy nhanh, rồi *trước khi tin nó*, bạn so với Jacobian tính bằng sai phân hữu hạn trên vài điểm ngẫu nhiên. Nếu sai số tương đối nhỏ thì dùng; nếu không thì bạn biết chắc lỗi nằm trong đúng khối vừa viết, chứ không phải ở solver.

Lan truyền ngược là cùng một việc, chỉ khác quy mô: một mạng n tầng là một hàm hợp, **lượt ngược** (*backward pass*) là chế độ nghịch của quy tắc chuỗi — đúng cái mà cộng đồng tối ưu gọi là *adjoint method* — và kiểm tra gradient là bước so Jacobian. Giai đoạn 1 vì thế không dạy bạn khái niệm mới; nó chuyển một kỹ năng bạn đã có sang miền mới, ở quy mô nhỏ tới mức mọi con số còn nhìn được bằng mắt.

B. Bước 1 — linear classifier với hai hàm mất mát

Bắt đầu ở chỗ đơn giản nhất mà vẫn còn thú vị: một phân loại tuyến tính $s = Wx + b$, cài cả *hinge loss* kiểu SVM lẫn **softmax loss** (*softmax cross-entropy loss*), viết cả **lượt xuôi** (*forward pass*) lẫn lượt ngược cho từng cái. Điều bạn sẽ thấy tận mắt — chứ không phải đọc thấy — là hai hàm này có hành vi *gradient khác nhau về bản chất*, không chỉ khác về công thức.

Ví dụ nhỏ nhất. Một mẫu, ba lớp, điểm số $s = (2,0; 0,5; -1,0)$, nhãn đúng là lớp 0.

- Hinge loss, lẽ $\Delta = 1$:

$$L = \sum_{j \neq y} \max(0, s_j - s_y + \Delta) = \max(0, -0,5) + \max(0, -2,0) = 0.$$

Loss bằng không, và *gradient cũng bằng đúng không* — mọi phần tử của $\partial L / \partial s$ là 0. Mẫu này đã thoả lẽ; nó không còn nói gì với mô hình nữa.

- Softmax loss, cùng bộ điểm số: $e^{2,0} = 7,389$, $e^{0,5} = 1,649$, $e^{-1,0} = 0,368$, tổng 9,406, nên $p = (0,786; 0,175; 0,039)$ và $L = -\ln 0,786 = 0,241$. Gradient có dạng đẹp đến mức đáng nhớ, $\partial L / \partial s = p - e_y$:

$$\partial L / \partial s = (-0,214; +0,175; +0,039).$$

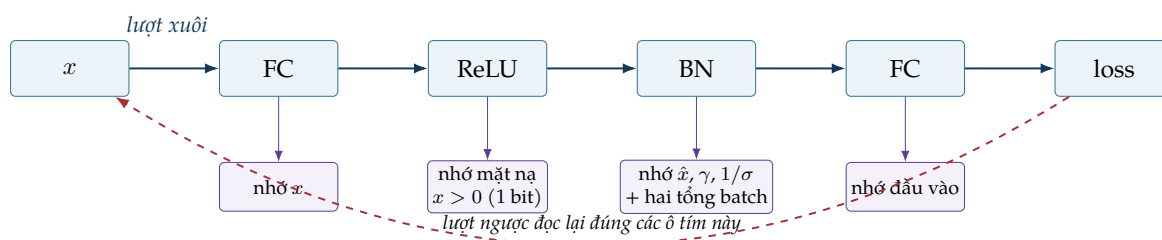
Ý nghĩa của công thức $p - e_y$: gradient chính là “sai lệch giữa niềm tin hiện tại và sự thật”. Không phần tử nào bằng không, nên softmax *luôn* còn đẩy — nó không có khái niệm “đủ tốt rồi”.

Khác biệt này giải thích rất nhiều hành vi thực nghiệm. Hinge tạo gradient thưa nên ổn định và ít bị chi phối bởi vài mẫu dễ, nhưng nó dừng học ngay khi lẽ được thoả, kể cả khi bạn còn muốn mô hình tự tin hơn. Softmax luôn có gradient nên tiếp tục tinh chỉnh, nhưng chính vì thế nó nhạy với nhãn sai — một mẫu bị gán nhầm sẽ bơm gradient vào suốt quá trình huấn luyện, không bao giờ tắt. Ai đã đọc B/09-thiet-ke-ham-mat-mat sẽ nhận ra đây là cùng trục lập luận đã dẫn tới *focal loss*.

→ cùng nguyên lý với B/09 — cùng trục lập luận đã dẫn tới focal loss — dời trọng số gradient khỏi những mẫu không còn dạy được gì.

C. Bước 2 — lượt ngược giải tích cho sáu tầng

Bước tiếp theo là viết lượt ngược giải tích cho *fully-connected*, ReLU, dropout, BatchNorm, *convolution* và *max-pooling*. Giá trị của bước này không nằm ở các công thức — chúng có sẵn ở mọi nơi — mà ở chỗ nó buộc bạn trả lời cho từng tầng một câu hỏi mà framework không bao giờ hỏi bạn: *backward của tầng này cần nhớ gì từ lượt xuôi?* Câu trả lời là toàn bộ lý do bộ nhớ huấn luyện lớn hơn bộ nhớ suy luận nhiều lần. Hình 1 vẽ đúng cơ chế đó.



Hình 1. Vì sao huấn luyện tốn bộ nhớ hơn suy luận: mỗi tầng phải *ghi lại* một phần trạng thái lúc lượt xuôi để lượt ngược đọc lại. Ô tím là thứ không giải phóng được cho tới khi lượt ngược chạy qua. ReLU rẻ (một bit mỗi phần tử), BatchNorm đắt nhất vì ngoài $\hat{x}$ nó còn ràng buộc cả *batch*.

Bảng 1. Sáu tầng, và thứ mà lượt ngược của mỗi tầng buộc phải giữ lại từ lượt xuôi.

Tầng	Backward cần nhớ	Chi phí	Chỗ hay sai
Fully-connected	Đầu vào x (cho $\partial L / \partial W$)	Bằng kích thước đầu vào	Quên chuyển vị; quên cộng dồn $\partial L / \partial b$ theo batch
ReLU	Chỉ mặt nạ $x > 0$ (1 bit mỗi phần tử)	Rất rẻ	Xử lý $x = 0$ tùy tiện — chính là điểm gây làm kiểm tra gradient báo động giả
Dropout	Mặt nạ đã dùng lúc lượt xuôi	Bằng kích thước đầu ra	Không dùng <i>inverted dropout</i> , hoặc sinh mặt nạ mới ở lượt ngược
BatchNorm	$\hat{x}, \gamma, 1/\sigma$ — và lượt ngược khớp cả <i>batch</i>	Đắt nhất trong sáu tầng	Coi lượt ngược là phép theo từng phần tử
Convolution	Ma trận <i>im2col</i> hoặc đầu vào để dựng lại nó	Rất đắt nếu vật chất hoá im2col	<i>col2im</i> phải cộng dồn vùng chồng lấn, không phải ghi đè
Max-pool	Chỉ số <i>argmax</i> của mỗi cửa sổ	Rẻ	Xử lý trường hợp bằng nhau khác giữa hai cài đặt

Ba dòng trong Bảng 1 đáng dừng lại.

BatchNorm — lượt ngược không phải phép theo từng phần tử

$$\frac{\partial L}{\partial x_i} = \frac{\gamma}{N\sigma} \left[N \frac{\partial L}{\partial y_i} - \sum_j \frac{\partial L}{\partial y_j} - \hat{x}_i \sum_j \frac{\partial L}{\partial y_j} \hat{x}_j \right].$$

Công thức này nói điều gì? Rằng gradient của mẫu i bị trừ đi hai đại lượng tính trên *toàn batch* — một cho trung bình, một cho phương sai. Nói cách khác: BatchNorm không cho phép một mẫu tự do đẩy gradient của nó; mọi mẫu trong batch cùng thoả hiệp. Ba hệ quả bạn sẽ gặp lại suốt sự nghiệp:

- **Bộ nhớ lượt ngược tăng** vì phải giữ cả $\hat{x}$ lẫn các thống kê, và không giải phóng được theo từng mẫu.
- **Batch nhỏ làm gradient nhiễu**, vì ước lượng σ nhiễu — lý do GroupNorm và LayerNorm tồn tại [1], [2].
- **Mô hình huấn luyện với BatchNorm là hàm của cả batch, không phải của từng mẫu** — lý do sâu xa khiến `train()` và `eval()` cho kết quả khác nhau.

Dropout — vì sao phải scale lúc train

Cài theo kiểu *inverted dropout*: lúc train, giữ lại phần tử với xác suất $1 - p$ rồi chia cho $1 - p$; lúc test không làm gì cả. Lý do là bảo toàn kỳ vọng — nếu $\tilde{y} = m \odot y / (1 - p)$ với $m \sim \text{Bernoulli}(1 - p)$ thì $\mathbb{E}[\tilde{y}] = y$, nên đường suy luận của các tầng sau không bị dịch khi ta bật tắt dropout. Cách còn lại (nhân $1 - p$ lúc test) tương đương về toán học nhưng dở về kỹ thuật, vì nó đặt một bước bắt buộc vào đường suy luận — và mọi đường inference đều có ngày bị viết lại, bị *export*, bị port sang C++, và bước ấy sẽ rơi mất. Một điểm trung thực cần ghi: *inverted dropout* chỉ khớp *mô-men bậc một*; phương sai lúc train vẫn lớn hơn lúc test, và đó là một phần lý do dropout tương tác xấu với BatchNorm [3].

Convolution — chỗ nào sinh ra bất định trên GPU

Cách cài dễ hiểu nhất là *im2col*: trải các cửa sổ thành cột rồi gọi một phép nhân ma trận, nên $\partial L / \partial W = \partial L / \partial Y \cdot X_{\text{col}}^\top$ và $\partial L / \partial X_{\text{col}} = W^\top \cdot \partial L / \partial Y$. Bước cuối — *col2im* — phải cộng dồn gradient vào những vị trí mà các cửa sổ chồng lên nhau. Ghi nhớ chi tiết này. Phép cộng dồn tán xạ đó chính là chỗ các cài đặt GPU dùng `atomicAdd`. Cộng số thực dấu phẩy động không có tính kết hợp, nên thứ tự luồng hoàn thành quyết định bit cuối. Đây là một trong những nguồn bất định chính của huấn luyện trên GPU, chủ đề của D/22-tai-lap-va-mlops. Nó cùng một họ nguyên nhân với các lỗi bất định do thứ tự con trỏ mà bạn đã gặp trong SLAM.

↪ **tiền đề** D/22 — cộng số thực không có tính kết hợp, nên mọi phép cộng dồn tán xạ trên GPU là một nguồn bất định — cùng họ nguyên nhân với thứ tự con trỏ trong một hệ thống C++ đa luồng.

D. Bước 3 — kiểm tra gradient, và kỷ luật đi kèm

Mọi công thức ở trên đều có thể sai, và sai một cách im lặng. Cách duy nhất phát hiện là so với sai phân trung tâm:

$$\frac{\partial f}{\partial x_k} \approx \frac{f(x + he_k) - f(x - he_k)}{2h},$$

Ta nhiễu một toạ độ về hai phía và xem loss thay đổi bao nhiêu. Rồi tính sai số tương đối $|g_{\text{số}} - g_{\text{giải tích}}| / \max(|g_{\text{số}}|, |g_{\text{giải tích}}|)$.

```
def grad_check(f, x, analytic, h=1e-5, n=20):
    # f: ham tra ve scalar loss; analytic: gradient da tinh giai tích
    idx = np.random.choice(x.size, size=n, replace=False)
```

```

for k in idx:
    old = x.flat[k]
    x.flat[k] = old + h; fp = f(x)
    x.flat[k] = old - h; fm = f(x)
    x.flat[k] = old
    num = (fp - fm) / (2 * h)
    ana = analytic.flat[k]
    rel = abs(num - ana) / max(1e-12, abs(num), abs(ana))
    print(k, num, ana, rel)

```

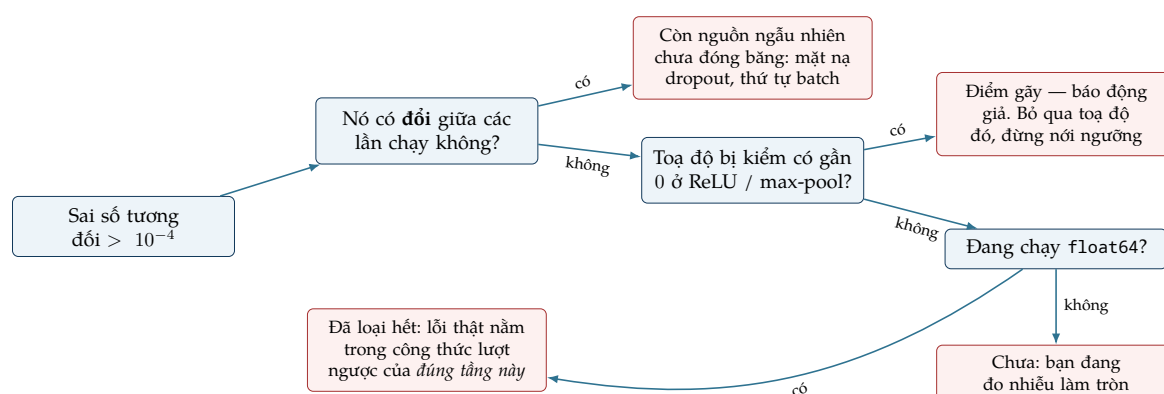
Ngưỡng diễn giải, theo *kinh nghiệm thực hành* chứ không phải kết quả lý thuyết:

- $< 10^{-7}$ — yên tâm.
- 10^{-7} đến 10^{-4} — chấp nhận được với mạng sâu có nhiều điểm gãy, đáng nghi với một tầng đơn giản.
- $> 10^{-4}$ — gần như chắc chắn có lỗi. $> 10^{-2}$ — chắc chắn có lỗi.

Ba chi tiết quyết định việc kiểm tra gradient nói thật hay nói dối:

1. **Phải dùng float64.** Sai phân trung tâm lấy hiệu hai số gần bằng nhau; với float32 (epsilon máy $\approx 1,2 \times 10^{-7}$) và $h = 10^{-5}$, bạn mất phần lớn chữ số có nghĩa vì mất chữ số có nghĩa khi trừ hai số gần bằng nhau — kết quả là bạn đang đo nhiễu làm tròn chứ không đo đạo hàm.
2. **Kiểm ở vài chục tọa độ ngẫu nhiên**, không phải toàn bộ tensor — vừa đủ tin cậy, vừa đủ nhanh để bạn thực sự chạy nó.
3. **Mọi nguồn ngẫu nhiên phải bị đóng băng giữa hai lần gọi f** ; nếu không, bạn đang so đạo hàm của hai hàm khác nhau.

Khi con số báo động, Hình 2 là thứ tự chẩn đoán tiết kiệm nhất.



Hình 2. Thứ tự chẩn đoán khi kiểm tra gradient báo động. Ba nhánh đầu là các nguyên nhân *giả*; chỉ khi loại hết chúng thì kết luận “công thức lượt ngược sai” mới có giá trị.

Kỷ luật đi kèm chỉ có một câu, và nó là phần đáng giá nhất của cả mục: *không ghép một tầng chưa qua kiểm tra*. Lý do thuần túy là số học tổ hợp. Ghép sáu tầng rồi mới phát hiện gradient sai thì không gian nghi vấn là sáu tầng cộng mọi tương tác giữa chúng; kiểm từng tầng ngay khi viết xong thì không gian nghi vấn luôn có đúng một phần tử. Đây chính xác là lập luận bạn đã dùng khi thêm từng loại ràng buộc vào một **đồ thị nhân tố** (*factor graph*) và kiểm tra sau mỗi lần thêm.

↔ **trường hợp riêng của A/01** — lan truyền ngược là chế độ nghịch của quy tắc chuỗi, nên kiểm tra gradient ở đây đúng là bước so Jacobian giải tích với Jacobian sai phân trong tối ưu phi tuyến.

E. Bước 4 — ghép thành mạng và tự viết vòng lặp

Bước cuối là ghép các tầng đã kiểm thành một mạng n tầng và tự viết vòng lặp huấn luyện với SGD kèm *momentum*. Ở đây bạn gặp một loại lỗi mới mà kiểm từng tầng không bắt được: lỗi về *trạng thái*.

- **Mặt nạ dropout** phải giống nhau giữa lượt xuôi và lượt ngược của cùng một bước, nhưng khác nhau giữa các bước.
- **BatchNorm** phải cập nhật thống kê chạy ở chế độ train và dùng thống kê ấy ở chế độ test.
- **Bộ đệm momentum** phải được cấp phát một lần, không phải mỗi bước.

Ba lỗi này là ba phiên bản của cùng một câu hỏi: *cái gì là tham số, cái gì là trạng thái, cái gì là tạm*. Chính câu hỏi đó quay lại nguyên vẹn ở mục sau, dưới cái tên parameter so với buffer.

F. Tiêu chí dừng, giới hạn và trường hợp hỏng

Tiêu chí dừng nên phát biểu bằng năng lực giải thích, không bằng số bài đã làm. Bạn dừng được khi giải thích trôi chảy hai điều. Thứ nhất, vì sao BatchNorm khiến lượt ngược tốn thêm bộ nhớ: hai tổng trên toàn batch, nên không giải phóng được theo từng mẫu. Thứ hai, vì sao dropout phải scale lúc train chứ không lúc test: để bảo toàn kỳ vọng mà không đặt bước bắt buộc nào lên đường suy luận. Đạt rồi thì chuyển sang PyTorch và không viết NumPy nữa.

Giới hạn của cả giai đoạn cần nói thẳng:

- **Mọi trực giác về hiệu năng rút ra từ đây đều sai.** NumPy thuần chậm hơn nhiều bậc so với kernel thật, nên bạn chỉ chạy được ở quy mô đồ chơi; phần hiệu năng phải học lại ở B/11-gioi-han-tinh-toan.
- **Không học được gì về ổn định số học ở độ chính xác thấp.** *Log-sum-exp*, *loss scaling*, tràn số fp16 chỉ xuất hiện khi bạn rời float64.
- **Kiểm tra gradient kiểm tính nhất quán, không kiểm tính đúng.** Một tầng cài sai công thức lượt xuôi nhưng có lượt ngược khớp với chính nó sẽ qua kiểm một cách hoàn hảo.

CẠM BÃY

Bốn cách làm cho kiểm tra gradient qua trong khi mã vẫn sai, xếp theo mức độ hay gặp. **Một:** quên cố định mặt nạ dropout, khiến $f(x + h)$ và $f(x - h)$ là hai hàm khác nhau — triệu chứng là sai số lớn và *nhảy loạn giữa các lần chạy*. **Hai:** để BatchNorm ở chế độ train trong lúc kiểm, khiến hàm bị kiểm phụ thuộc thống kê batch và thay đổi khi bạn nhiễu một toạ độ. **Ba:** kiểm ở đúng điểm gãy của ReLU hoặc max-pool, tức x rất gần 0. Ở đó sai phân hai phía đo qua một chỗ hàm không khả vi, nên báo động là giả. Cách xử lý đúng là bỏ qua toạ độ đó, không phải nới ngưỡng. **Bốn:** chạy ở float32 rồi kết luận rằng “sai số 10^{-3} là bình thường”; không, đó là dấu hiệu bạn đang đo nhiễu làm tròn.

CẦN NHỚ

Không ghép một tầng chưa qua kiểm tra gradient. Lý do không phải sự cẩn thận mà là tổ hợp: kiểm từng tầng giữ không gian nghi vấn ở đúng một phần tử; ghép trước rồi kiểm sau làm nó phình lên thành số tầng nhân với số tương tác. Và nhớ rằng kiểm tra gradient chỉ chứng minh lượt xuôi và lượt ngược *khớp nhau*, không chứng minh lượt xuôi đúng. [T2]

G. Kết nối

Mục này là tiền đề bắt buộc của D/19/02-pytorch-thanh-thao. Ba khái niệm bạn vừa cài bằng tay — đồ thị tính toán, thứ phải nhớ giữa hai lượt, và tham số so với trạng thái — sẽ xuất hiện lại dưới dạng autograd, nn.Module và cặp `train()/eval()`. Bạn sẽ hiểu chúng như tên gọi mới của thứ đã biết. Nó là trường hợp riêng của A/01-toan-toi-thieu/02-vi-phan-va-toi-uu: backpropagation chỉ là chế độ nghịch của quy tắc chuỗi áp cho hàm hợp [4]. Nó bổ sung cho C/12-co-che-huan-luyen: chương kia nói BatchNorm và dropout *làm gì* cho việc huấn luyện, mục này nói chúng *tồn gì* và hỏng ở đâu. Và nó gieo trước một hạt cho D/22-tai-lap-va-mlops: phép cộng dồn tán xạ trong *col2im* là nguồn bất định GPU mà bạn sẽ gặp lại ở đó.

■ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao hinge loss cho gradient bằng đúng không ở một mẫu đã thoả lề, còn softmax thì không bao giờ? Hệ quả nào của điều đó xuất hiện khi tập dữ liệu có nhãn sai?
2. Hai tổng trên toàn batch trong lượt ngược của BatchNorm là gì, và chúng buộc ta giữ thêm gì trong bộ nhớ?
3. Vì sao *inverted dropout* tốt hơn về mặt kỹ thuật dù hai cách scale tương đương về toán học? Và nó *không* bảo toàn thứ gì?
4. Kiểm tra gradient cho sai số tương đối 3×10^{-3} và con số đó đổi mỗi lần chạy. Nguyên nhân khả dĩ nhất là gì, và nếu con số ổn định thì danh sách nghi vấn đổi thế nào?
5. Kiểm tra gradient qua hết mọi tầng nhưng mạng vẫn không học — hai loại lỗi nào mà kiểm tra gradient theo thiết kế không thể bắt được?
6. Chỗ nào trong lượt ngược của convolution là phép cộng dồn tán xạ, và vì sao chi tiết đó sẽ quay lại ám bạn khi chạy trên GPU?

Nguồn

- [1] Y. Wu and K. He, “Group Normalization,” in *ECCV*, 2018.
- [2] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” *arXiv:1607.06450*, 2016.
- [3] N. Srivastava, G. Hinton, A. Krizhevsky, et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *JMLR*, vol. 15, pp. 1929–1958, 2014.
- [4] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.